

RASTRO

Transparencia y Datos Abiertos Colombia — Hackathon Croma

ARQUITECTURA DEL SISTEMA

Arquitectura del Sistema

Vista de contenedores, patrón de puertos y adaptadores, resiliencia y despliegue.

Versión 1.0 — Agosto de 2026

Proyecto: RASTRO — motor de transparencia en contratación pública colombiana

1. Vista general

RASTRO se compone de dos procesos independientes que se despliegan por separado: un backend Express que expone una API REST pura (sin servir HTML), y un frontend Next.js que renderiza la interfaz y actúa como proxy autenticado hacia esa API. El backend nunca es alcanzado directamente por el navegador — todas las llamadas del cliente van a rutas propias `/api/\*` del frontend, que reenvían la petición al backend agregando las cabeceras de autenticación necesarias.

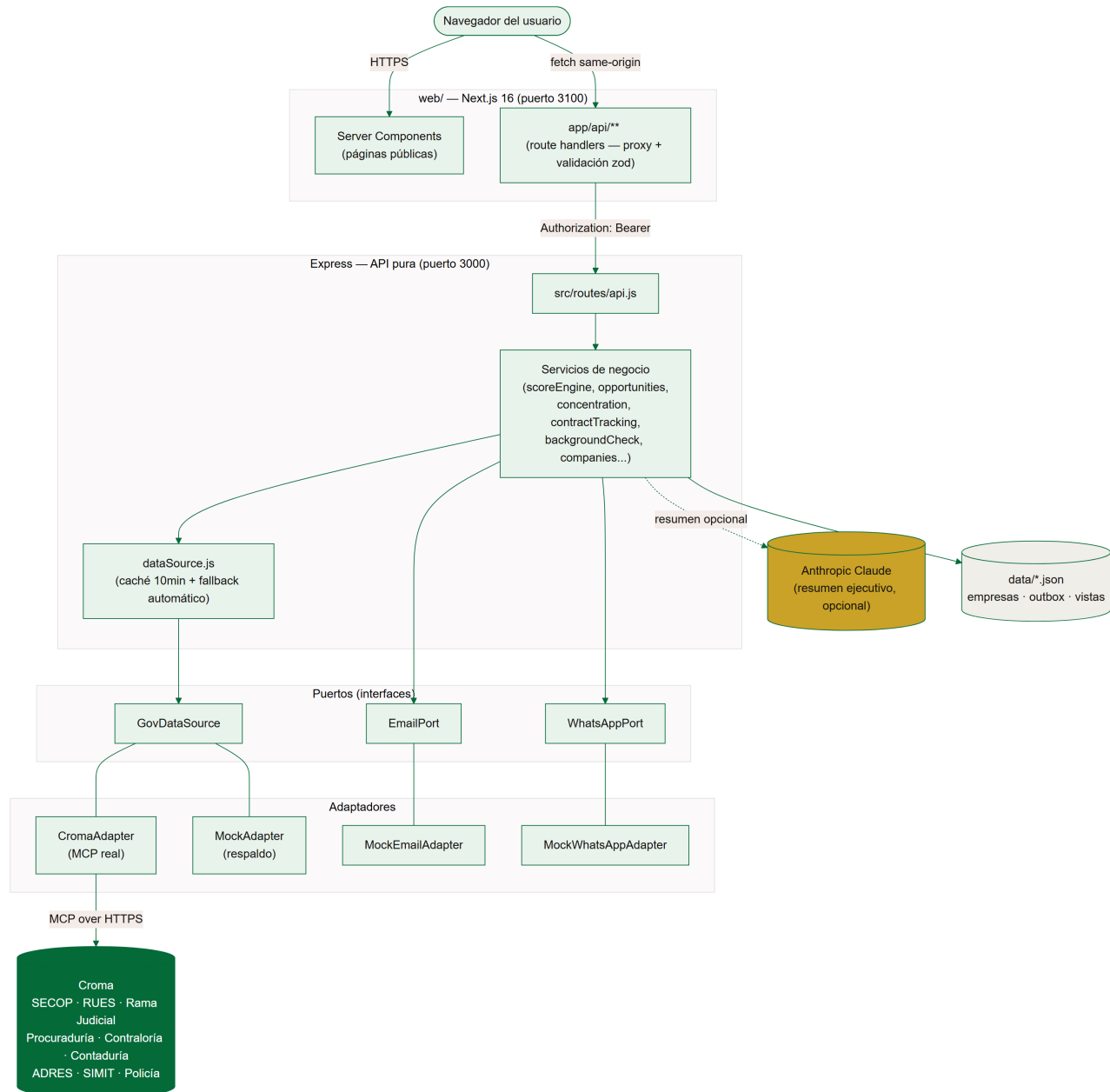


Figura 1 — Vista de contenedores de RASTRO.

2. Patrón de puertos y adaptadores

El núcleo de negocio (servicios en ``src/services/``) nunca depende de una implementación concreta de acceso a datos externos — depende de interfaces ("puertos"): **GovDataSource** para las fuentes oficiales, **EmailPort** para el envío de correo y **WhatsAppPort** para WhatsApp. Cada puerto tiene al menos dos adaptadores: uno real (Croma) y uno simulado (Mock), que implementan exactamente la misma interfaz.

Esto tiene dos consecuencias directas en el comportamiento del producto: primero, se puede reemplazar por completo la fuente de datos, el proveedor de correo o el de WhatsApp cambiando una variable de entorno y un archivo de adaptador nuevo, sin tocar ningún servicio de negocio. Segundo, y más importante en producción, permite degradar automáticamente ante una falla externa sin que el resto del sistema se entere.

2.1 Composición y resiliencia — ``dataSource.js``

Es el único punto donde se decide qué adaptador de *GovDataSource* está activo. Envuelve cada uno de sus 13 métodos con dos capas:

- **Caché en memoria** con TTL de 10 minutos, por método y argumentos — evita golpear a Croma con la misma consulta repetida durante una sesión de uso o una demo.
- **Degradación automática:** si la llamada al adaptador primario (Croma) falla, se reintenta una vez con una conexión nueva y, si sigue fallando, cae de forma transparente al adaptador de respaldo (Mock) — sin lanzar error hacia el servicio que la llamó.

El estado de salud se calcula sobre una ventana móvil de las últimas 12 llamadas de cualquier método: si más de la mitad de esas llamadas cayeron a respaldo, ``source.degraded`` se reporta como verdadero — visible en ``GET /api/meta`` para que el frontend (y el equipo) sepan en qué modo está operando el sistema en cada momento.

2.2 Adaptadores simulados como ciudadanos de primera clase

Los adaptadores Mock no son solo un recurso de pruebas: son la razón por la que el producto nunca se cae por completo. ``MockAdapter`` responde con un conjunto curado de entidades, oportunidades y contratos de ejemplo. ``MockEmailAdapter`` y ``MockWhatsAppAdapter`` registran cada envío en ``data/outbox_emails.json`` / ``data/outbox_whatsapp.json`` en vez de contactar un proveedor real, y devuelven el código/mensaje generado directamente en la respuesta de la API cuando no hay un proveedor real configurado — marcado explícitamente como "modo demo" en la interfaz.

3. Módulos del backend

Módulo	Responsabilidad
search.js	Búsqueda y cascada de normalización de nombre → NIT/cédula.
scoreEngine.js	Construye el expediente de riesgo cruzando 6 fuentes en paralelo.
contractTracking.js	Ejecución financiera, línea de tiempo y señal de alineación de un contrato.
opportunities.js	Licitaciones activas por entidades semilla, filtros y ganadores anteriores.
concentration.js	Concentración de adjudicaciones recientes por proveedor, con datos reales.
backgroundCheck.js	Dossier de persona natural (estudio de seguridad), acceso restringido.

Módulo	Responsabilidad
companies.js	Cuentas de empresa: registro, login en dos pasos, suscripción, verificación de WhatsApp.
notifications.js	Sondeo periódico de oportunidades nuevas y disparo de alertas por correo.
aiSummary.js	Resumen ejecutivo narrativo con Claude, nunca un veredicto.
evidence.js / normalize.js	Utilidades compartidas de clasificación de evidencia y normalización de texto.

4. Persistencia

RASTRO es mayormente *stateless*: la información de contratación se recalcula en cada consulta contra Croma (con caché de corta duración). Lo que sí necesita sobrevivir a un reinicio del proceso se guarda en archivos JSON planos (`src/store/jsonStore.js`), suficiente para el volumen de un hackathon/demo:

- `data/companies.json` — cuentas de empresa, suscripciones, estado de verificación de WhatsApp.
- `data/outbox_emails.json` y `data/outbox_whatsapp.json` — historial de envíos simulados.
- `data/seen_opportunities.json` — oportunidades ya notificadas, para no alertar dos veces.

El camino natural de evolución si el volumen de empresas registradas lo justifica es migrar esta capa a una base de datos relacional (Postgres), sin cambiar la interfaz que ya usan los servicios.

5. Despliegue

Los dos procesos se despliegan y escalan de forma independiente. El backend (`server.js`) corre como un proceso Node de larga duración (necesario para mantener viva la conexión MCP persistente con Croma y el sondeo periódico de oportunidades). El frontend (`web/`) es una aplicación Next.js estándar, desplegable en Vercel, que se comunica con el backend a través de la variable de entorno `RASTRO_API_URL`.